

6 PORTFOLIO PROJECTS EVERY CS STUDENT NEEDS

Build these. Get hired.

Full breakdowns, architecture diagrams, and resource links

By @1nska

NYU Shanghai — 2026

Why Your Portfolio Matters More Than Your GPA

Here's what most CS programs won't tell you: the students landing the best jobs aren't the ones with the highest grades. They're the ones who can point to something they built and say "I made this, it's live, and here's the GitHub repo."

A to-do app won't cut it. Neither will a calculator or a tutorial clone. Recruiters have seen thousands of those. What they haven't seen is a student who built a real-time system, designed a distributed architecture, or published an open source tool that other developers actually use.

These 6 projects cover the key areas top tech companies look for: full-stack development, system design, AI/ML integration, distributed systems, open source contribution, and CS fundamentals. Build 2-3 of these and you'll stand out from 90% of applicants.

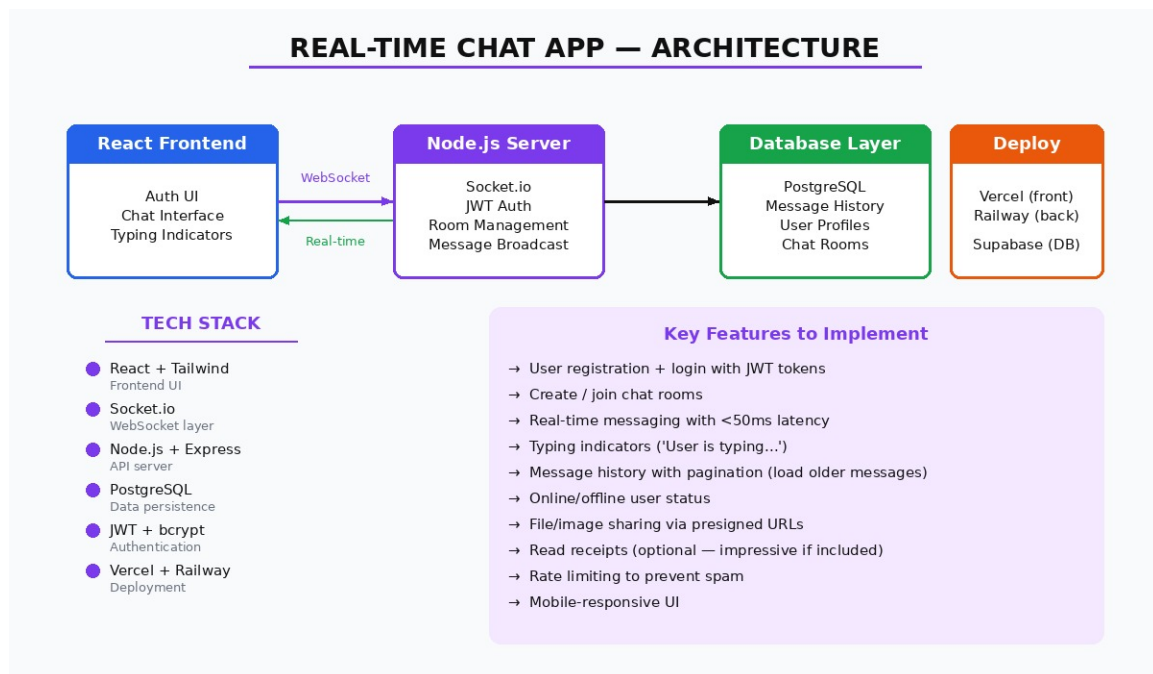
How to use this guide: Each project includes architecture diagrams, step-by-step build instructions, and clickable resource links. Start with whichever excites you most — momentum beats perfection.

#1

Real-Time Chat App with WebSockets

FULL-STACK • NETWORKING • REAL-TIME

Difficulty: Intermediate | Estimated time: 2-3 weekends



Why recruiters love this: Every tech company has real-time features — notifications, live updates, collaborative editing. Building a chat app from scratch shows you understand WebSockets, state management, authentication, and full-stack architecture.

Build a fully functional chat application with user authentication, multiple chat rooms, real-time messaging, typing indicators, and message history. Not a tutorial clone — a production-grade app with proper error handling, deployed and accessible via a live URL.

Tech stack:

- **React + Tailwind CSS** — frontend UI with responsive design
- **Node.js + Express** — backend API server
- **Socket.io** — WebSocket layer for real-time communication
- **PostgreSQL + Prisma** — database with ORM for message history and users
- **JWT + bcrypt** — authentication and password hashing
- **Vercel + Railway** — deployment (both free tier)

Skills it proves:

- WebSocket protocol and real-time event handling
- Full-stack architecture — frontend, backend, database, deployment
- Authentication and security (JWT, password hashing, input sanitization)
- Database design and query optimization
- State management across client and server

How to build it:

Step 1: Set up the project structure. Initialize a monorepo with separate frontend (React) and backend (Node.js) directories. Set up Prisma with PostgreSQL. Define your data models: User, Room, Message.

Step 2: Build authentication. Implement signup/login with bcrypt password hashing and JWT tokens. Add middleware to protect API routes. Store tokens in httpOnly cookies for security.

Step 3: Implement WebSocket layer. Set up Socket.io on the server. Create events: 'join-room', 'send-message', 'typing', 'stop-typing'. Connect the React frontend to the socket server.

Step 4: Build the chat UI. Create the room list, message feed, and input components. Messages should appear instantly for the sender and within 50ms for other users. Add typing indicators.

Step 5: Add message persistence. Save every message to PostgreSQL. Implement pagination — load the 50 most recent messages on room join, then load older messages on scroll up.

Step 6: Deploy and polish. Deploy frontend to Vercel, backend to Railway, database to Supabase. Add error handling, loading states, and a clean landing page. Get a live URL you can share.

Useful links & resources:

- [Socket.io documentation](#)
- [Prisma ORM guide](#)
- [JWT authentication tutorial](#)
- [Railway deployment guide](#)
- [React real-time chat tutorial — freeCodeCamp](#)

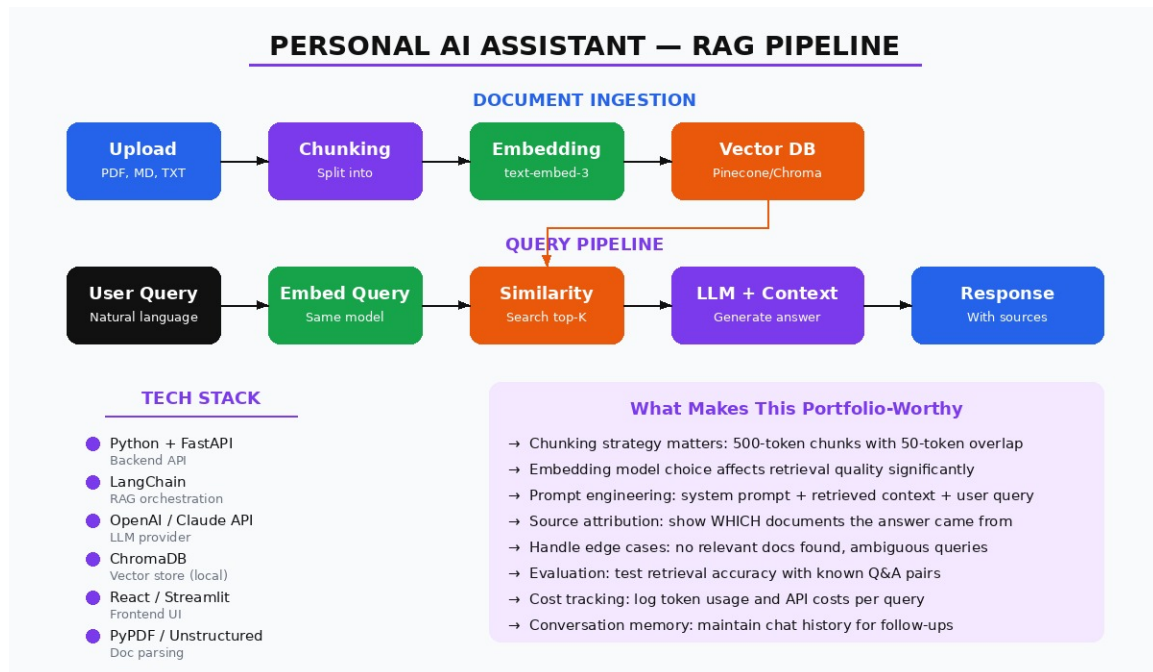
Level up: Add end-to-end encryption using the Web Crypto API so messages are encrypted client-side before being sent. This is a serious security feature that will make any interviewer take notice — and it's a great talking point about cryptography.

#2

Personal AI Assistant with RAG

AI / ML • NLP • FULL-STACK

Difficulty: Intermediate–Advanced | **Estimated time:** 2-3 weekends



Why recruiters love this: Every company is integrating AI right now. But calling the ChatGPT API isn't impressive — building a RAG pipeline that retrieves from your own data, manages context windows, and provides sourced answers shows you actually understand how AI applications work under the hood.

Build an application that lets users upload their own documents (PDFs, notes, articles) and have a conversation with them. The system chunks documents, creates embeddings, stores them in a vector database, retrieves relevant chunks for each query, and generates answers with source attribution.

Tech stack:

- **Python + FastAPI** — backend API
- **LangChain or LlamaIndex** — RAG orchestration framework
- **OpenAI / Anthropic API** — LLM for answer generation
- **ChromaDB or Pinecone** — vector database for embeddings
- **React or Streamlit** — frontend interface
- **PyPDF / Unstructured** — document parsing and extraction

Skills it proves:

- RAG architecture — document ingestion, embedding, retrieval, generation
- Vector databases and similarity search
- Prompt engineering and context window management
- API design for AI-powered applications
- Evaluation — measuring retrieval quality and answer accuracy

How to build it:

Step 1: Set up document ingestion. Build an upload endpoint that accepts PDFs and text files. Use PyPDF or Unstructured to extract text. Implement chunking — 500-token chunks with 50-token overlap works well as a starting point.

Step 2: Create the embedding pipeline. Use OpenAI's text-embedding-3-small (cheap and effective) to embed each chunk. Store embeddings in ChromaDB (local, free) with metadata linking back to the source document and page number.

Step 3: Build the retrieval system. When a user asks a question, embed the query with the same model. Search ChromaDB for the top 5 most similar chunks. These become the context for the LLM.

Step 4: Implement the generation step. Build a prompt template: system instructions + retrieved chunks + user question. Send to Claude or GPT-4. Parse the response and include source attributions — which document and page the answer came from.

Step 5: Add conversation memory. Store chat history and include the last 3-5 exchanges in each prompt. This lets users ask follow-up questions like "tell me more about that" without losing context.

Step 6: Build the frontend and deploy. Create a clean chat interface with a document upload panel. Show source documents alongside answers. Deploy the backend to Railway, frontend to Vercel. Demo it with your own class notes or textbooks.

Useful links & resources:

- [LangChain RAG tutorial](#)
- [ChromaDB quickstart](#)
- [OpenAI embeddings guide](#)
- [Anthropic Claude API docs](#)
- [RAG evaluation with RAGAS](#)
- [LlamaIndex starter tutorial](#)

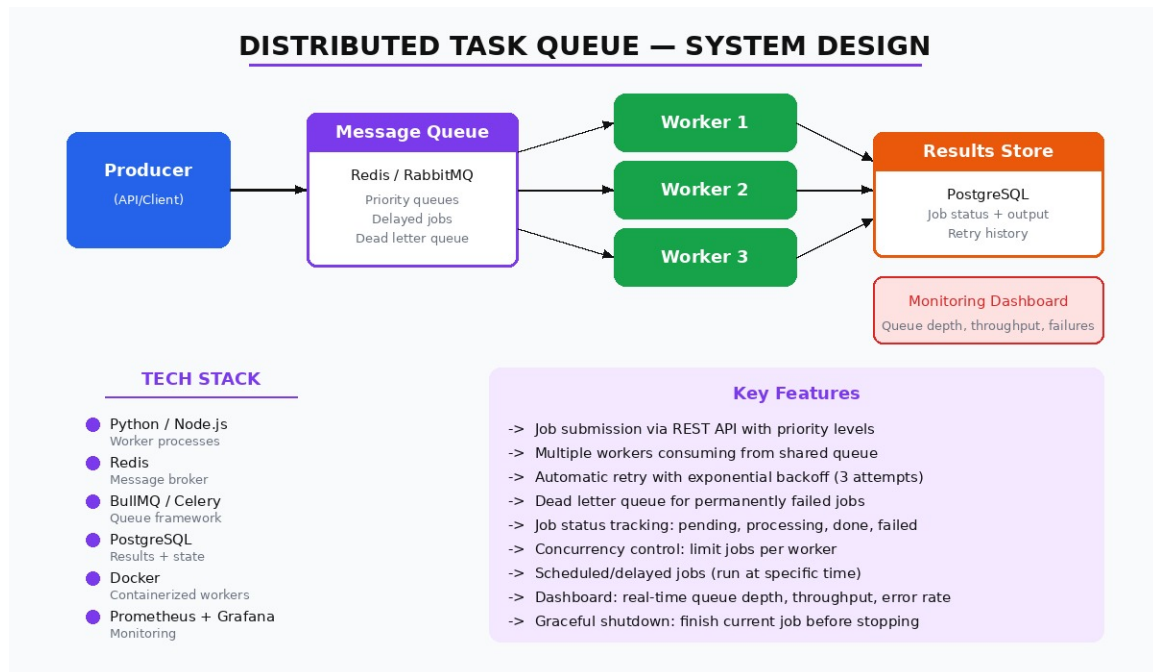
Level up: Add multi-modal support — let users upload images and use a vision model to extract information from diagrams, charts, and handwritten notes. Or implement hybrid search (keyword + semantic) for better retrieval accuracy.

#3

Distributed Task Queue System

SYSTEMS • DISTRIBUTED • BACKEND

Difficulty: Advanced | Estimated time: 3-4 weekends



Why recruiters love this: This is the kind of project that makes senior engineers raise an eyebrow. Distributed systems are the backbone of every tech company at scale — understanding job queues, worker management, retry logic, and failure handling shows you think about infrastructure, not just features.

Build a distributed task processing system where clients submit jobs via an API, a message queue distributes them across multiple worker processes, and a monitoring dashboard shows real-time throughput, queue depth, and failure rates. Include retry with exponential backoff, dead letter queues, and priority levels.

Tech stack:

- **Python (Celery) or Node.js (BullMQ)** — worker framework
- **Redis** — message broker and job queue
- **PostgreSQL** — results store and job history
- **FastAPI or Express** — REST API for job submission
- **Docker + Docker Compose** — containerized workers for true distribution

- **Prometheus + Grafana** — monitoring and dashboards (or build your own)

Skills it proves:

- Distributed systems design — message passing, eventual consistency
- Queue theory — FIFO, priority queues, dead letter handling
- Fault tolerance — retry strategies, idempotency, graceful shutdown
- Containerization and orchestration with Docker
- Monitoring and observability — metrics, logging, alerting

How to build it:

Step 1: Design the job schema. Define what a 'job' looks like: ID, type, payload, priority, status (pending/processing/done/failed), attempts, created_at, completed_at. Store this in PostgreSQL.

Step 2: Set up Redis as the message broker. Use Redis lists or streams for the job queue. Implement priority queues — high-priority jobs should be processed before low-priority ones even if they arrived later.

Step 3: Build the producer API. Create a REST endpoint that accepts job submissions, validates the payload, writes to the database, and pushes to the Redis queue. Return a job ID for status polling.

Step 4: Build the worker process. Create a worker that continuously polls Redis, picks up a job, processes it, and updates the database with results. Implement graceful shutdown — finish the current job before stopping.

Step 5: Add retry logic. When a job fails, retry up to 3 times with exponential backoff (1s, 4s, 16s). After 3 failures, move to the dead letter queue. Log every attempt with the error message.

Step 6: Build the monitoring dashboard. Show real-time queue depth, jobs/second throughput, success/failure rate, and average processing time. Use Grafana or build a simple React dashboard that polls a stats endpoint.

Useful links & resources:

- [BullMQ documentation](#)
- [Celery project documentation](#)
- [Redis documentation](#)
- [Docker Compose tutorial](#)
- [Designing Data-Intensive Applications \(book\)](#)
- [System design primer — GitHub](#)

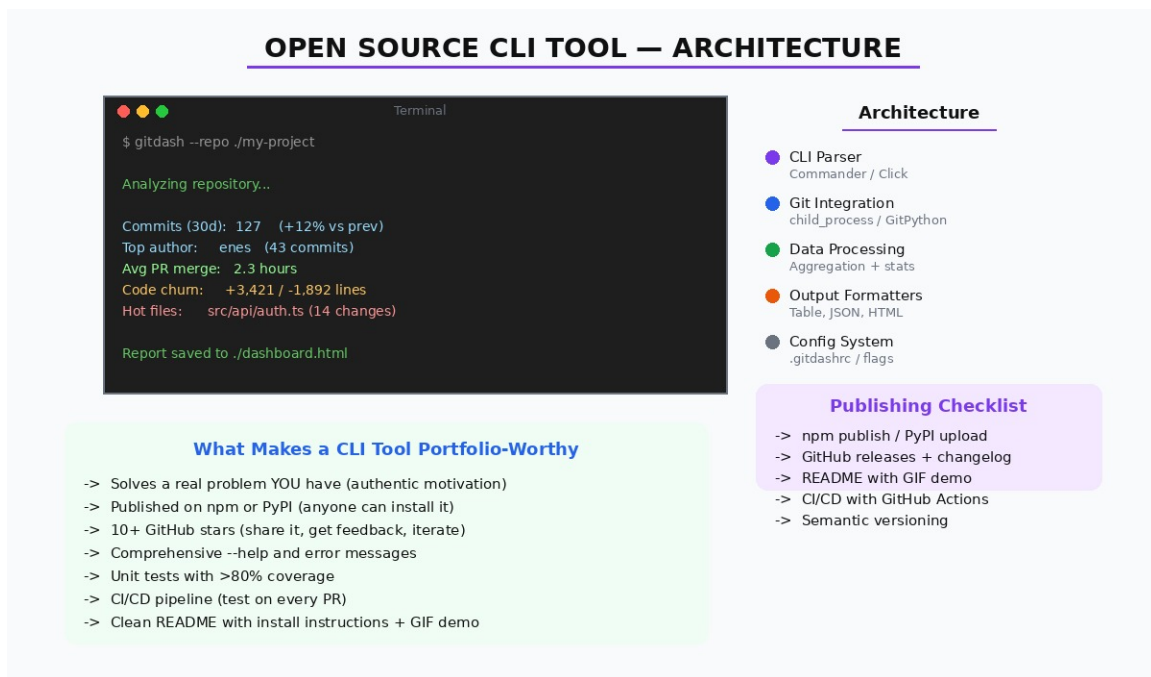
Level up: Add horizontal auto-scaling — spin up new worker containers when queue depth exceeds a threshold and scale down when idle. Use Kubernetes or Docker Swarm. This is exactly what companies like Uber, Stripe, and Shopify do at scale.

#4

Published Open Source CLI Tool

OPEN SOURCE • DEVELOPER TOOLS • PUBLISHING

Difficulty: Intermediate | Estimated time: 1-2 weekends



Why recruiters love this: Open source contributions show you can write code that other people actually use, read, and contribute to. Publishing a CLI tool on npm or PyPI with proper documentation, tests, and CI/CD demonstrates professional software engineering practices that most students never learn.

Build a command-line tool that solves a real problem developers have. Publish it on npm or PyPI so anyone can install it with one command. Get real users, handle bug reports, and iterate based on feedback. The tool should have a clean README with a GIF demo, comprehensive --help output, and unit tests.

Tech stack:

- **Node.js (Commander.js) or Python (Click/Typer)** — CLI framework
- **Jest or pytest** — unit testing
- **GitHub Actions** — CI/CD pipeline
- **npm or PyPI** — package registry for publishing
- **Chalk / Rich** — terminal formatting and colors

- **Semantic versioning** — proper release management

Skills it proves:

- Open source software development — README, LICENSE, CONTRIBUTING.md
- Package publishing and distribution
- Test-driven development with high coverage
- CI/CD automation with GitHub Actions
- User-facing documentation and error handling

How to build it:

Step 1: Find your problem. The best CLI tools come from scratching your own itch. Ideas: a git stats dashboard, a project scaffolder, a markdown-to-slides converter, an API testing tool, a dependency auditor, a code complexity analyzer.

Step 2: Build the core functionality. Use Commander.js (Node) or Click/Typer (Python) for argument parsing. Focus on doing one thing well before adding features. Make sure errors are helpful, not cryptic.

Step 3: Add comprehensive tests. Write unit tests for every major function. Aim for 80%+ coverage. Set up GitHub Actions to run tests on every push and PR. Add a coverage badge to your README.

Step 4: Write documentation. Your README needs: what it does (one sentence), install instructions, usage examples with screenshots/GIFs, API reference for all flags and commands. Use asciinema or terminalizer for GIF recording.

Step 5: Publish to npm/PyPI. For npm: `npm publish`. For PyPI: `twine upload`. Set up automated publishing via GitHub Actions on new releases. Use semantic versioning — start at 0.1.0.

Step 6: Get users and iterate. Share on Reddit (r/commandline, r/webdev), Hacker News, Twitter/X. Respond to issues. Accept PRs. Even 10 GitHub stars puts you ahead of 99% of student portfolios.

Useful links & resources:

- [Commander.js documentation](#)
- [Python Click documentation](#)
- [How to publish an npm package](#)
- [PyPI publishing guide](#)
- [GitHub Actions CI/CD tutorial](#)
- [asciinema — terminal recording](#)

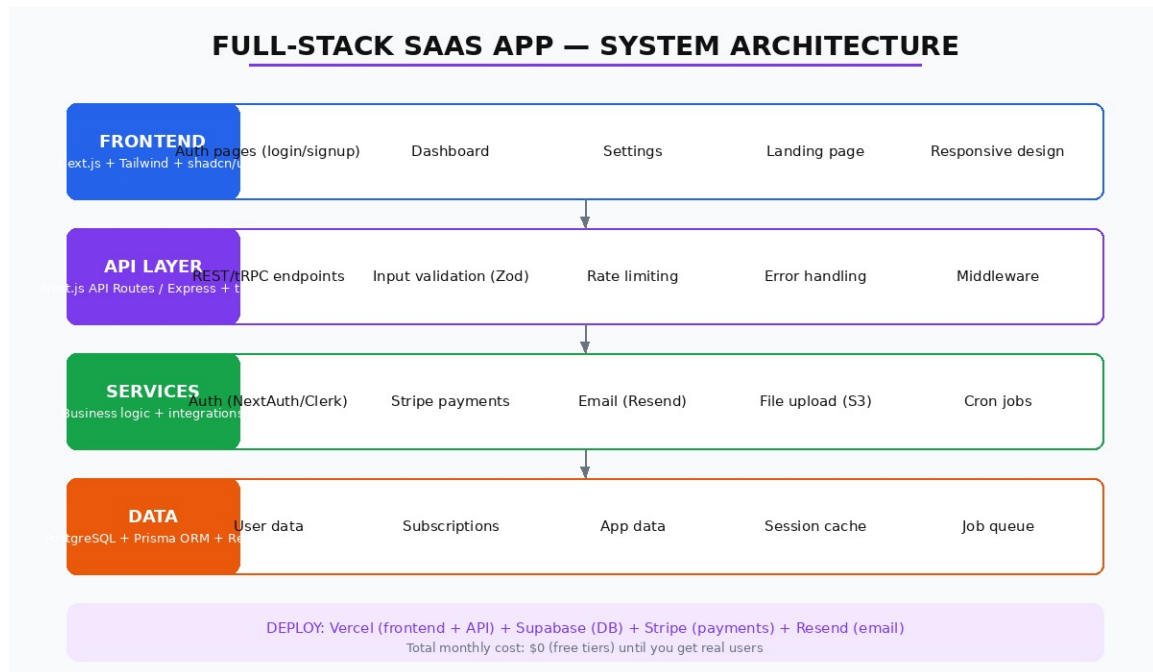
Level up: Add plugin support so other developers can extend your tool. This shows API design thinking and turns your project into a platform. Tools like ESLint, Prettier, and Babel all work this way.

#5

Full-Stack SaaS Application

FULL-STACK • PRODUCT • PAYMENTS

Difficulty: Advanced | Estimated time: 4-6 weekends



Why recruiters love this: Building a SaaS app proves you can think beyond code — you understand users, products, and business logic. Adding Stripe payments, user onboarding, and a landing page shows full-product thinking that most CS students never demonstrate.

Build a complete SaaS application with user authentication, a dashboard, Stripe subscription payments, email notifications, and a marketing landing page. Pick any useful idea — a habit tracker, invoice generator, bookmark manager, study planner, writing tool. The specific idea matters less than the architecture and polish.

Tech stack:

- **Next.js 14** — full-stack React framework (app router)
- **Tailwind CSS + shadcn/ui** — UI components and styling
- **NextAuth.js or Clerk** — authentication (Google, GitHub, email login)
- **PostgreSQL + Prisma** — database with type-safe ORM
- **Stripe** — subscription payments and billing
- **Resend** — transactional emails (welcome, receipts, notifications)

Skills it proves:

- Product thinking — user flows, onboarding, retention
- Payment integration — Stripe checkout, webhooks, subscription lifecycle
- Authentication patterns — OAuth, session management, role-based access
- Database design for multi-tenant SaaS
- Full deployment and DevOps — CI/CD, monitoring, environment management

How to build it:

Step 1: Start with auth and database. Set up Next.js with NextAuth or Clerk. Configure PostgreSQL with Prisma. Build signup, login, and a protected dashboard route. This is the foundation everything else builds on.

Step 2: Build the core feature. Whatever your SaaS does — build the main feature with CRUD operations. Use server actions or API routes. Make sure it's genuinely useful, even if simple. Test it yourself for a week.

Step 3: Add Stripe payments. Implement a pricing page with free/pro tiers. Set up Stripe Checkout for subscriptions. Handle webhooks for payment events (subscription created, updated, canceled). Gate pro features behind payment status.

Step 4: Build the landing page. Create a marketing page with: clear value proposition, feature list, pricing table, social proof (even if it's just 'built by a student at NYU'), and a CTA button. This page should convert visitors to signups.

Step 5: Add email notifications. Set up Resend for transactional emails: welcome email on signup, payment receipt, and a weekly digest. These small touches make the product feel professional.

Step 6: Deploy and launch. Deploy to Vercel (frontend + API) + Supabase (DB). Set up a custom domain if possible. Share on Product Hunt, Reddit, and LinkedIn. Track analytics with Vercel Analytics or Plausible.

Useful links & resources:

- [Next.js 14 documentation](#)
- [Stripe integration guide](#)
- [NextAuth.js documentation](#)
- [shadcn/ui component library](#)
- [Resend email API](#)
- [SaaS starter template — Open SaaS](#)

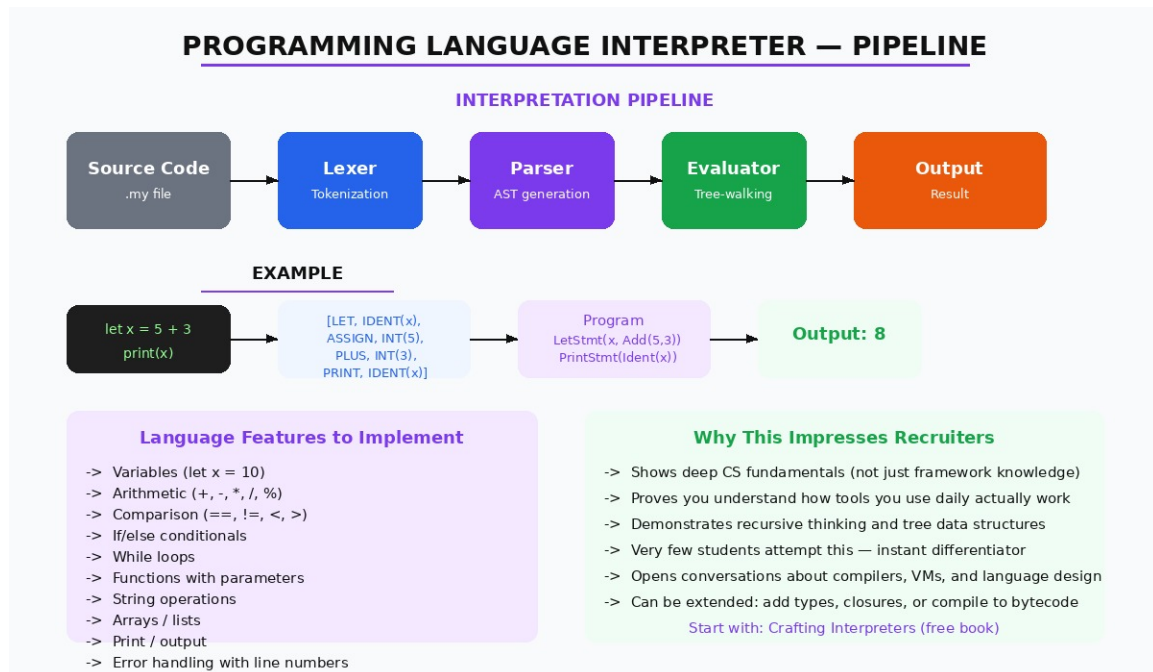
Level up: Add AI-powered features using the OpenAI or Anthropic API. For example, if it's a writing tool, add AI suggestions. If it's an invoice tool, add smart categorization. AI features in a SaaS app show you're building with current technology.

#6

Programming Language Interpreter

CS FUNDAMENTALS • COMPILERS • LANGUAGE DESIGN

Difficulty: Advanced | Estimated time: 3-5 weekends



Why recruiters love this: This is the project that makes engineers say "wait, you built that as a student?" Writing an interpreter proves deep CS fundamentals — parsing, tree traversal, scope management, type systems. Very few students attempt this, which makes it an instant differentiator.

Build a tree-walking interpreter for a simple programming language you design. It should support variables, arithmetic, conditionals, loops, functions with parameters, and print output. Write it in Python, JavaScript, or Go. Include a REPL (interactive console) and the ability to run script files.

Tech stack:

- **Python, JavaScript, or Go** — implementation language (pick one you know well)
- **No frameworks** — build the lexer, parser, and evaluator from scratch
- **pytest / Jest** — testing each stage of the pipeline
- **Crafting Interpreters (free book)** — the definitive guide
- **GitHub Pages** — host an interactive playground for your language

Skills it proves:

- Lexical analysis — tokenization, regular expressions
- Parsing — recursive descent, Abstract Syntax Trees (ASTs)
- Evaluation — tree-walking interpretation, environment/scope management
- Language design — syntax decisions, operator precedence, error reporting
- Software engineering — clean architecture, comprehensive testing

How to build it:

Step 1: Build the lexer (tokenizer). Read source code character by character. Output a stream of tokens: NUMBER, STRING, IDENTIFIER, PLUS, MINUS, LET, IF, FUNCTION, etc. Handle edge cases like multi-character operators (`==`, `!=`) and string escaping.

Step 2: Build the parser. Take the token stream and produce an Abstract Syntax Tree (AST). Use recursive descent parsing. Start with expressions (math), then add statements (let, print), then control flow (if/else, while), then functions.

Step 3: Build the evaluator. Walk the AST and execute each node. Implement an Environment class for variable scoping. Handle nested scopes (functions should have their own scope that can access parent scope).

Step 4: Add functions. This is where it gets interesting. Functions need parameter binding, local scope, return values, and (optionally) closures. Test with recursive functions like fibonacci and factorial.

Step 5: Add error handling with line numbers. When something goes wrong (undefined variable, type mismatch), show the exact line number and a helpful error message. Good errors separate a toy from a real tool.

Step 6: Build the REPL and file runner. Create an interactive mode where users type expressions and see results immediately. Also support running `.mylang` script files. Publish on GitHub with example programs and a playground.

Useful links & resources:

- [Crafting Interpreters — free online book](#)
- [Writing An Interpreter In Go \(book\)](#)
- [Build your own Lisp — free guide](#)
- [AST Explorer — visualize syntax trees](#)
- [Pratt parsing explained](#)
- [Language implementation patterns \(book\)](#)

Level up: Compile your language to bytecode and build a virtual machine to execute it. This is what Python, Ruby, and Lua do internally. Or add a type system with type inference — this shows you understand concepts most CS grads only see in theory.

Quick Reference

#	Project	Key Skill Area	Difficulty	Time
1	Real-Time Chat	Full-Stack + Networking	Intermediate	2-3 weekends
2	RAG AI Assistant	AI/ML + NLP	Intermediate	2-3 weekends
3	Task Queue	Distributed Systems	Advanced	3-4 weekends
4	CLI Tool	Open Source + DevOps	Intermediate	1-2 weekends
5	SaaS App	Full-Stack + Product	Advanced	4-6 weekends
6	Interpreter	CS Fundamentals	Advanced	3-5 weekends

What To Do Next

Don't try to build all 6. Pick the one that excites you most, set up the repo today, and commit your first code this weekend. A finished project beats a perfect plan every time.

When you finish, package it properly: clean code on GitHub, a detailed README with screenshots or a demo GIF, a live deployment link, and test coverage. That package is what gets you interviews at companies most students think are out of reach.

Essential free tools for CS students:

- [Vercel](#) — deploy frontend + API for free
- [Railway](#) — deploy backends for free
- [Supabase](#) — free PostgreSQL database
- [GitHub](#) — portfolio hosting, CI/CD, version control
- [Cursor](#) — AI-powered code editor

Follow @1nska on Instagram for more project guides, build logs, and real talk about building as a student. DM me "BUILD" and I'll help you pick the right project for your skill level.

— Enes (@1nska)
NYU Shanghai, 2026